MEDAC
Instituto Oficial de Formación Profesional

Mi primera app Android: de la idea al proyecto con recursos

Jose Luis Mba Bechiro Mifumu

Índice

Mi primera app Android: de la idea al proyecto con recursos **1**
1. Análisis del desarrollo móvil **3**
1.1.- Ventajas e inconvenientes 3
1.2.- Elección tecnológica: Tabla comparativa 5
1.3 Ciclo de vida de un activity 6
2. Especificación de la app **6**
2.1.- Idea y público objetivo 6
2.2.- Objetivos de la app 6
2.3.- Requisitos funcionales 7
2.4.- Requisitos no funcionales 7
3. Entorno de desarrollo **8**
4. Evidencias visuales de ejecución 10
4.1.- Configuración Multi Idioma 10
4.2.- Evidencias visuales de ejecución 11
5. Control de Versiones y Despliegue en GitHub **12**
6. Conclusion Personal **12**

1. Análisis del desarrollo móvil

1.1.- Ventajas e inconvenientes

El desarrollo de aplicaciones para dispositivos móviles se ha convertido en un pilar fundamental de la ingeniería de software actual. En comparación con el desarrollo de escritorio o web tradicional, los entornos móviles ofrecen ventajas inigualables, como la portabilidad de aprovechar componentes de hardware nativos (cámara, GPS, sensores de movimiento) que enriquecen la experiencia de usuario de forma directa.

No obstante, desarrollar para smartphones y tablets conlleva enfrentarse a un conjunto de restricciones técnicas muy severas que no existen o tienen un impacto mínimo en un ordenador de escritorio conectado a la red eléctrica. A continuación, se analizan 4 limitaciones concretas y cómo afectan específicamente a nuestra app ControlGastos:

a) **Gestión de la batería y consumo energético**

Limitación: los dispositivos móviles dependen de una batería de capacidad finita. Procesos en segundo plano descontrolados o peticiones intensivas de red descargan el terminal con rapidez.

Afección en nuestra app: ControlGastos es una utilidad local ligera. Para evitar drenar la batería del usuario, la aplicación no mantiene servicios pesados en segundo plano ni sincronizaciones constantes con servidores remotos; opera de forma local y eficiente, activando recursos únicamente cuando el usuario interactúa activamente con la pantalla.

b) **Memória RAM limitada**

Limitación: aunque los terminales modernos han ganado capacidad, la gestión de procesos del sistema operativo móvil (Low Memory Killer) es muy agresiva para liberar memoria de forma constante.

Afección en nuestra app: para prevenir cierres inesperados (crashes) o ralentizaciones, nuestra app prescinde de animaciones complejas o carga masiva de

elementos gráficos pesados en la memoria RAM, optimizando la estructura visual mediante contenedores ligeros en XML.

c) Conectividad inestable o intermitente

Limitación: a diferencia de un ordenador de escritorio con conexión por cable fija, un dispositivo móvil se desplaza constantemente, enfrentados a pérdidas de cobertura, zonas de sombra o transiciones entre redes Wi-Fi y datos móviles (4G/5G).

Afección en nuestra app: al estar diseñada para registrar pequeños gastos de forma inmediata, ControlGastos no requiere una conexión a Internet obligatoria en su funcionamiento básico, garantizando que el usuario pueda operar de manera fluida y offline en cualquier momento y lugar.

d) Restricciones de tamaño del tamaño de pantalla y ergonomía

Limitación: la superficie útil de visualización es reducida en comparación con un monitor de escritorio, lo que limita la cantidad de información que se puede mostrar simultáneamente sin saturar al usuario.

Afección en nuestra app:obliga a un diseño minimalista y centrado en la usabilidad táctil (Mobile First). Utilizaremos botones amplios, tipografías legibles con unidades sp y márgenes con unidades dp para que la introducción de datos de los gastos se realice de forma cómoda con una sola mano.

1.2.- Elección tecnológica: Tabla comparativa

Para abordar la construcción de la aplicación, es necesario analizar las tres grandes corrientes de desarrollo móvil existentes:

Caracteristica	**Desarrollo Nativo (Android/iOS)**	**Desarrollo Híbrido (Ionic/Cordova)**	**Desarrollo Multiplataforma (Flutter/React Native)**
Lenguajes	Java/Kotlin (Android), Swift / Objective-C (iOS)	HTML, CSS, JavaScript	Dart (Flutter) JavaScript/TypeScript (React Native)
Rendimiento	Óptimo y fluido, acceso directo al hardware sin intermedios.	Moderado; se ejecuta sobre un contenedor web (WebView.)	Alto; compite de cerca con el nativo mediante renderizado propio
Acceso al Hardware	Total y nativo a través de APIs oficiales del sistema operativo	Limitado; depende de plugins de terceros para acceder a sensores	Amplio; mediante puentes de comunicación (bridges) o plugins oficiales.
Coste de Desarrollo	Elevado si se desarrollan ambas plataformas por separado (requiere equipos distintos)	Bajo; un único código base sirve para múltiples plataformas.	Medio-Bajo; un único código base compilable a múltiples plataformas.
Ejemplo de Framework	Android Studio (para Android) (Xcode (para iOS).	Ionic Framework	Flutter (Google) React Native (Meta)

Justificación de la elección: Para la realización de esta práctica, desarrollaremos la aplicación utilizando un framework nativo en Android (Android

estudio, empleando Java y vistas en XML). Los motivos principales son dos: por un lado, nos permite cumplir rigurosamente con los contenidos y directrices académicas del módulo de Programación Multimedia y Dispositivos Móviles, adquiriendo una base sólida y profunda sobre el funcionamiento interno de Android; por otro lado, el desarrollo nativo garantiza el

máximo rendimiento posible, una interacción perfecta con los recursos del sistema y un control absoluto sobre la gestión de layouts y ficheros de recursos.

1.3 Ciclo de vida de un activity

2. Especificación de la app

2.1.- Idea y público objetivo

La aplicación ControlGastos es una herramienta móvil intuitiva y ligera diseñada para facilitar el registro rápido y visual de los pequeños desembolsos económicos cotidianos (como transporte, alimentación o compras menores del día a día). Su público objetivo abarca a un perfil amplio de usuarios desde estudiantes hasta trabajadores que necesitan llevar un control financiero básico, directo y sin las complicaciones mi fricciones que suponen las aplicaciones bancarias o de contabilidad avanzada. La interfaz prioriza un diseño minimalista, botones amplios y una navegación centrada en una única pantalla optimizada para el uso ágil con una sola mano.

2.2.- Objetivos de la app

Rendimiento: conseguir que la pantalla principal cargue y muestre todos sus elementos visuales en un tiempo inferior a 1.5 segundos tras iniciar la aplicación en el dispositivo.

Usabilidad: Permitir que el usuario pueda introducir y registrar un nuevo gasto completando un máximo de 2 interacciones efectivas (pulsaciones de botones) desde la pantalla principal.

Compatibilidad: garantizar la correcta visualización e independencia de los elementos de la interfaz sin solapamiento en al menos dos densidades de pantalla diferentes (dispositivos de densidad HDPI y XHDP).

2.3.- Requisitos funcionales

RF-01 (Prioridad Alta): El sistema debe permitir al usuario introducir una descripción corta o concepto del gasto en un campo de texto dedicado.

RF-02 (Prioridad Alta): El sistema debe permitir introducir la cantidad numérica exacta del gasto a través de un campo con teclado numérico optimizado.

RF-03 (Prioridad Media): El sistema debe mostrar un botón interactivo para validar y registrar el gasto introducido, reflejando de manera inmediata en la interfaz.

RF-04 (Prioridad Media): El sistema debe calcular y actualizar de forma automática el importe total acumulado de todos los gastos registrados en la sesión.

RF-05 (Prioridad Baja): El sistema debe incluir un botón de limpieza rápida para restablecer los campos de entrada y vaciar el acumulador a cero de forma instantánea.

2.4.- Requisitos no funcionales

RNF-01 (Rendimiento): El tiempo de respuesta de la interfaz al pulsar el botón de registrar gasto no superará los 100 ms bajo condiciones normales de ejecución en el emulador.

RNF-02 (Compatibilidad): La aplicación será compatible y se instalará correctamente en dispositivos con Android 7.0 (API nivel 24) o superior, asegurando soporte para la gran mayoría del parque actual de terminales móviles.

RNF-03 (Usabilidad): Todos los tamaños de texto se definirán utilizando exclusivamente unidades escalables independientes de la escala tipográfica (sp) y los márgenes y dimensiones en unidades independientes de densidad (dp), prohibiendo el uso de píxeles absolutos (px).

RMF-04 (Mantenibilidad): Ningún texto, color o dimensión estará escrito de forma directa o “a mano” (handcoded) en los layouts XML; todos se externalizan obligatoriamente en sus respectivos ficheros de recursos (strings.xml, colors.xml).

3. Entorno de desarrollo

La estructura de directorios y ficheros del proyecto se organiza de forma modular para separar claramente la lógica del código fuente y los recursos estéticos.

- manifests/ (AndroidManifest.xml): Es el fichero de configuración global de la aplicación. Declara los componentes principales (como la activity principal), los permisos requeridos del sistema y el icono de la aplicación que se mostrará en el lanzador del dispositivo.
- java/ : Contiene el código fuente de la aplicación organizando por paquetes (es.medac.joseluismba.app). Aquí reside la clase lógica principal (por ejemplo, MainActivity.java), encargada de gestionar el comportamiento de la pantalla.
- res/ (Recursos): Es el directorio crítico donde se centralizan los elemento visuales y no el código de la app, divididos en subcarpetas específicas:

  drawable/ : Imágenes estéticas y vectores (Vector Assets).

  layout/ : Ficheros de diseño de interfaz escritos en XML (ej: activity_main.xml).

  mipmap/ : Iconos adaptativos de la aplicación para las diferentes densidades de pantalla.

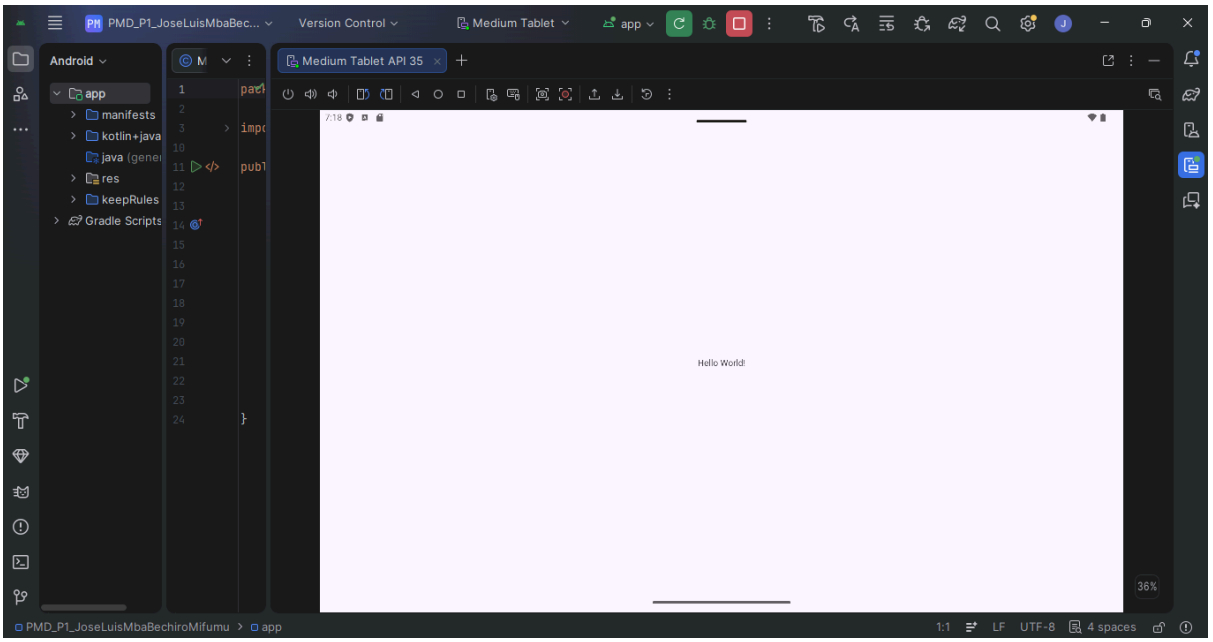
  values/ : Ficheros de configuración fundamental como strings.xml (textos), colors.xml (paleta de colores) y styles.xml (estilos visuales).

- Ficheros Gradle (Gradle Scripts): Se encargan de la compilación y gestión de dependencias del proyecto. Destacan el fichero a nivel de proyecto y el de modulo (build.gradle.kts o build.gradle), donde se definen las versiones decompilacion (compileSdk yy minSdk).

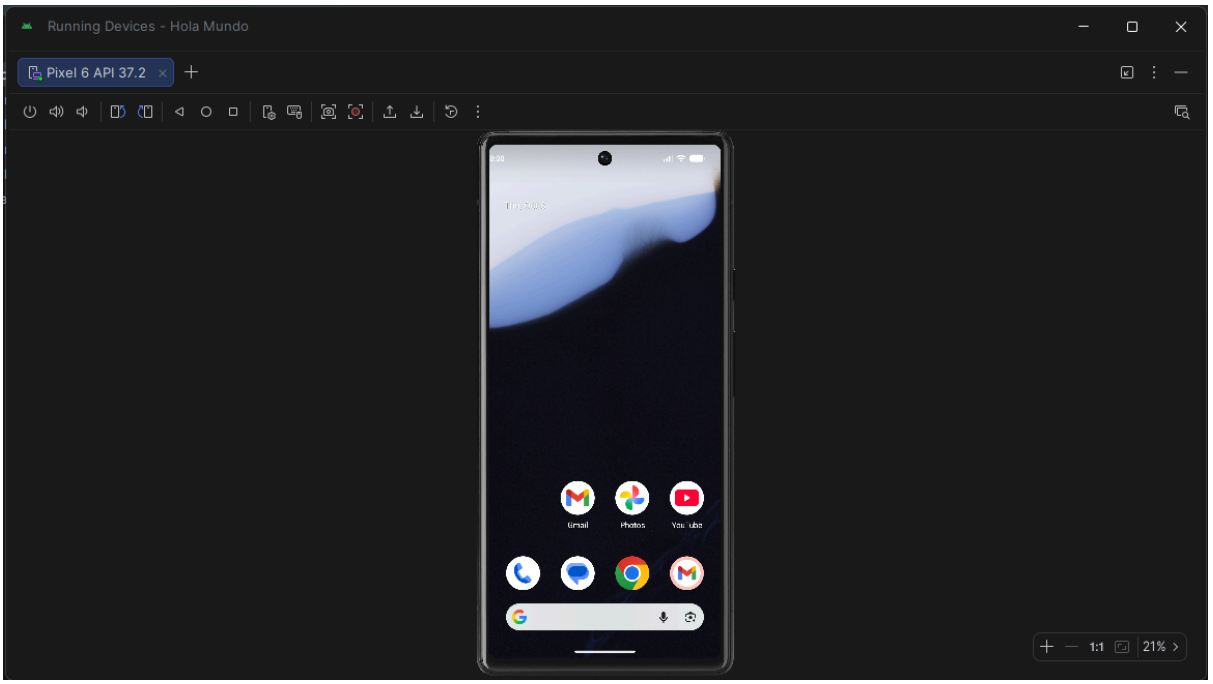
Relación entre la Activity (Java) y su Layout (XML)

En Android se aplica el principio de separación entre la estructura visual y la lógica de programación. El Layout (XML) ubicado en res/layout/ define como se ve la pantalla (botones, campos de texto, distribuciones), mientras que la Activity (Java) ubicada en java/ define como se comporta (las acciones al hacer clic, validaciones y eventos). La conexión entre ambos mundos se realiza mediante el método setContentView(R.layout.activity_main) dentro del ciclo de vida onCreate() de la Activity, inflando los elementos visuales para que puedan ser controlados desde el código Java.

Captura de pantalla de la tablet



Captura de pantalla del móvil

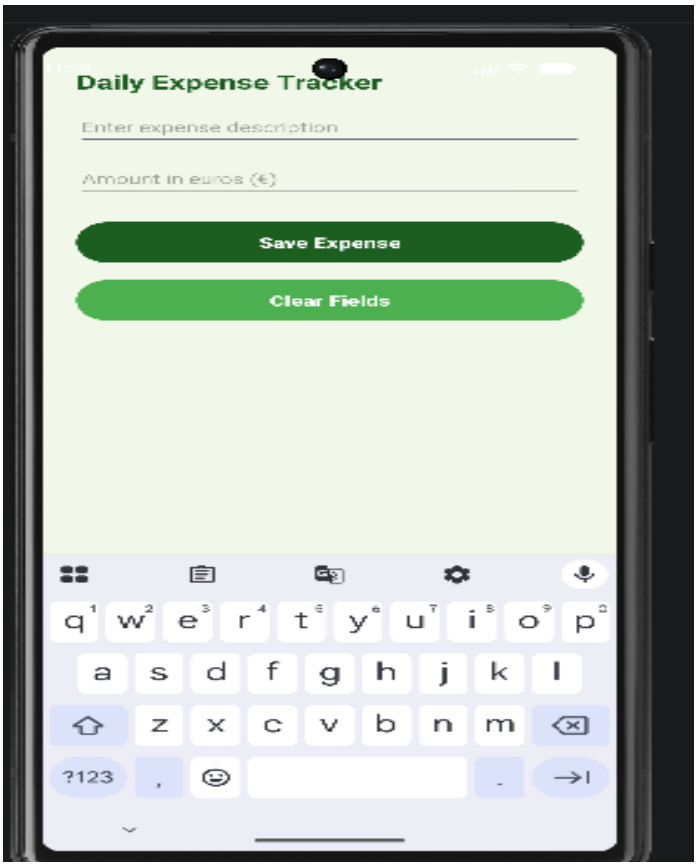


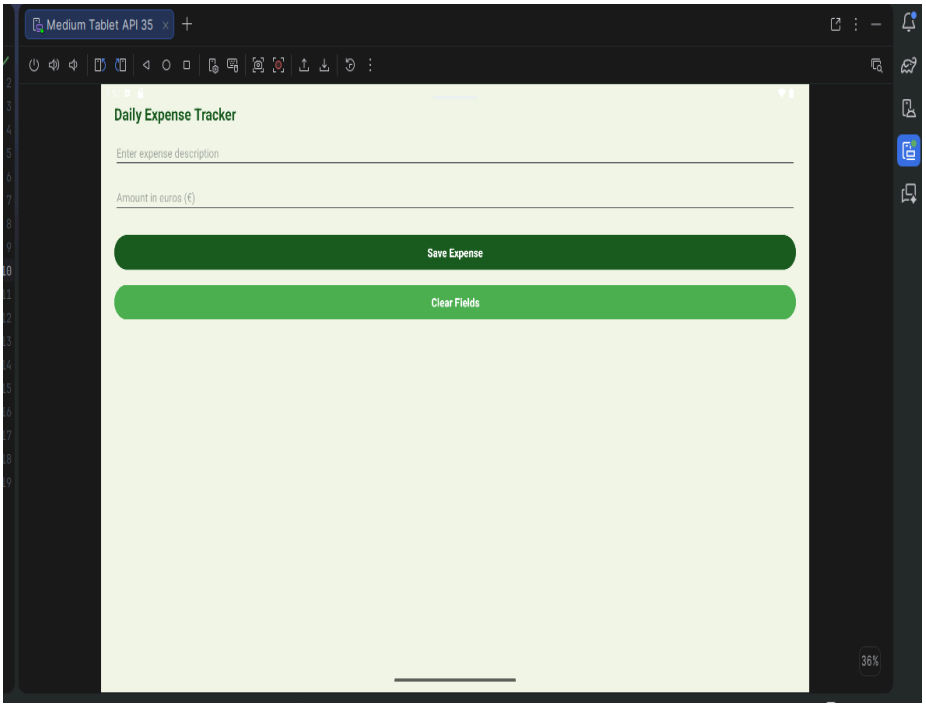
4. Evidencias visuales de ejecución

4.1.- Configuración Multi Idioma

Para dar cumplimiento a la internacionalización, se creó el directorio específico de recursos localizados res/values-en/ , dotándolo de su correspondiente fichero string.xml traducido al inglés. Al modificar el idioma del sistema en el emulador (Pixel 6), el sistema operativo mapea automáticamente los recursos textuales sin necesidad de alterar la lógica de la aplicación.

4.2.- Evidencias visuales de ejecución





5. Control de Versiones y Despliegue en GitHub

El control de cambios se ha llevado a cabo mediante Git, estructurando el historial de desarrollo con commits semánticos y descriptivos. El proyecto se encuentra alojado públicamente en el repositorio:

https://github.com/joseluismba60-ops/PMD_P1_ControlGastos.git

6. Conclusion Personal

El desarrollo de la aplicación ControlGastos ha permitido afianzar los conocimientos teóricos y prácticos sobre el ecosistema de Android Studio. La resolución de retos técnicos relacionados con la gestión de layouts, la externalización de recursos, el control de versiones con Git y la internacionalización ha resultado fundamental para comprender el flujo completo de creación de software para dispositivos móviles bajo estándares profesionales.